

Desafio — Cadastro de Alunos

```
# Importa o dataclass para criar uma estrutura de dados
from dataclasses import dataclass

# Cria a estrutura Aluno
@dataclass
class Aluno:

    # Guarda o nome do aluno
    nome: str

    # Guarda a idade do aluno
    idade: int

    # Guarda as três notas em uma lista
    notas: list[float]

# Cria uma lista vazia para armazenar os alunos
alunos = []

# Repete o cadastro 5 vezes
for i in range(5):

    # Mostra qual aluno está sendo cadastrado
    print(f"\nAluno {i + 1}")

    # Pedir o nome do aluno
    nome = input("Nome: ")

    # Pedir a idade do aluno
    idade = int(input("Idade: "))

    # Cria uma lista vazia para guardar as notas
    notas = []

    # Repete 3 vezes para receber as três notas
    for j in range(3):

        # Pedir uma nota ao usuário
        nota = float(input(f"Nota {j + 1}: "))
```

```
# Adiciona a nota na lista
notas.append(nota)

# Cria um objeto Aluno com os dados informados
aluno = Aluno(nome, idade, notas)

# Adiciona o aluno na lista de alunos
alunos.append(aluno)

# Guarda a maior média encontrada
maior_media = 0

# Guarda o aluno que possui a maior média
melhor_aluno = None

print("\nMédias dos alunos")

# Percorre todos os alunos cadastrados
for aluno in alunos:

    # Soma as três notas e divide por 3
    media = sum(aluno.notas) / 3

    # Mostra o nome e a média
    print(f"{aluno.nome} - Média: {media:.2f}")

    # Verifica se a média atual é maior que a maior encontrada
    if media > maior_media:

        # Atualiza a maior média
        maior_media = media

        # Guarda o aluno com a maior média
        melhor_aluno = aluno

# Mostra o aluno com a maior média
print("\nAluno com maior média")
print(f"Nome: {melhor_aluno.nome}")
print(f"Média: {maior_media:.2f}")
```

Exercício 1 — Soma de um vetor

```
# Cria uma lista vazia para armazenar os números
numeros = []

# Repete 10 vezes
for i in range(10):

    # Pede um número inteiro ao usuário
    numero = int(input(f"Digite o {i + 1}º número: "))

    # Adiciona o número na lista
    numeros.append(numero)

# Calcula a soma de todos os números
soma = sum(numeros)

# Calcula a média
# len() informa quantos números existem na lista
media = soma / len(numeros)

# Mostra os números digitados
print("\nNúmeros digitados")
print(numeros)

# Mostra a soma
print(f"Soma: {soma}")

# Mostra a média com duas casas decimais
print(f"Média: {media:.2f}")
```

Exercício 2 — Maior e menor

```
# Cria uma lista vazia
numeros = []

# Recebe 10 números
```

```
for i in range(10):

    # Pede um número inteiro
    numero = int(input(f"Digite o {i + 1}º número: "))

    # Adiciona o número à lista
    numeros.append(numero)

# Encontra o maior número da lista
maior = max(numeros)

# Encontra o menor número da lista
menor = min(numeros)

# Mostra o vetor
print("\nVetor")
print(numeros)

# Mostra o maior e o menor
print(f"\nMaior valor: {maior}")
print(f"Menor valor: {menor}")

# Cria uma lista para guardar as posições do maior
posicoes_maior = []

# Cria uma lista para guardar as posições do menor
posicoes_menor = []

# Percorre todas as posições da lista
for i in range(len(numeros)):

    # Verifica se o número atual é igual ao maior
    if numeros[i] == maior:

        # Guarda a posição
        posicoes_maior.append(i)

    # Verifica se o número atual é igual ao menor
```

```
if numeros[i] == menor:

    # Guarda a posição
    posicoes_menor.append(i)

# Mostra as posições encontradas
print(f"Posições do maior: {posicoes_maior}")
print(f"Posições do menor: {posicoes_menor}")
```

Exercício 3 — Números pares

```
# Cria uma lista vazia
numeros = []

# Recebe 20 números
for i in range(20):

    # Pede um número inteiro
    numero = int(input(f"Digite o {i + 1}º número: "))

    # Adiciona o número na lista
    numeros.append(numero)

# Guarda a quantidade de números pares
quantidade_pares = 0

# Guarda a soma dos números pares
soma_pares = 0

print("\nNúmeros pares")

# Percorre todos os números
for numero in numeros:

    # O operador % calcula o resto da divisão
    # Se o resto da divisão por 2 for 0, o número é par
    if numero % 2 == 0:

        # Mostra o número par
```

```
# Mostra o número par
print(numero)

# Aumenta a quantidade de pares
quantidade_pares += 1

# Adiciona o número à soma
soma_pares += numero

# Mostra a quantidade de números pares
print(f"\nQuantidade de pares: {quantidade_pares}")

# Mostra a soma dos números pares
print(f"Soma dos pares: {soma_pares}")
```

Exercício 4 — Inversão de vetor

```
# Cria uma lista vazia
numeros = []

# Recebe 10 números
for i in range(10):

    # Pede um número inteiro
    numero = int(input(f"Digite o {i + 1}º número: "))

    # Adiciona o número à lista
    numeros.append(numero)

# Mostra o vetor antes da inversão
print("\nVetor original")
print(numeros)

# Define a primeira posição
inicio = 0

# Define a última posição
fim = len(numeros) - 1
```

```
# Continua enquanto o início estiver antes do fim
while inicio < fim:

    # Troca o primeiro elemento pelo último
    numeros[inicio], numeros[fim] = numeros[fim], numeros[inicio]

    # Avança uma posição no início
    inicio += 1

    # Volta uma posição no final
    fim -= 1

# Mostra o vetor depois da inversão
print("\nVetor invertido")
print(numeros)
```

Exercício 5 — Matriz 3×3

```
# Cria uma matriz vazia
matriz = []

# Cria 3 linhas
for i in range(3):

    # Cria uma linha vazia
    linha = []

    # Cria 3 colunas
    for j in range(3):

        # Pede o valor daquela posição
        valor = int(input(f"Digite o valor [{i}][{j}]: "))

        # Adiciona o valor à linha
        linha.append(valor)

    # Adiciona a linha completa à matriz
    matriz.append(linha)

# Começa a soma em zero
soma = 0
```

```
# Considera inicialmente o primeiro elemento como o maior
maior = matriz[0][0]

# Percorre as linhas
for i in range(3):

    # Percorre as colunas
    for j in range(3):

        # Adiciona o elemento à soma
        soma += matriz[i][j]

        # Verifica se o elemento é maior que o atual maior
        if matriz[i][j] > maior:

            # Atualiza o maior valor
            maior = matriz[i][j]

# Mostra a matriz
print("\nMatriz")

for linha in matriz:

    # Mostra cada linha
    print(linha)

# Mostra a soma dos elementos
print(f"\nSoma: {soma}")

# Mostra o maior elemento
print(f"Maior valor: {maior}")
```

Exercício 6 — Diagonal principal

```
# Cria uma matriz vazia
matriz = []
```

```
# Cria 4 linhas
```



```
# Cria 4 linhas
for i in range(4):

    # Cria uma linha vazia
    linha = []

    # Cria 4 colunas
    for j in range(4):

        # Pede o valor ao usuário
        valor = int(input(f"Digite o valor [{i}][{j}]: "))

        # Adiciona o valor à linha
        linha.append(valor)

    # Adiciona a linha à matriz
    matriz.append(linha)

# Começa a soma da diagonal em zero
soma = 0

print("\nDiagonal principal")

# Percorre os índices da matriz
for i in range(4):

    # Na diagonal principal,
    # o índice da linha é igual ao índice da coluna
    valor = matriz[i][i]

    # Mostra o valor da diagonal
    print(valor, end=" ")

    # Adiciona o valor à soma
    soma += valor

# Mostra a soma da diagonal
print(f"\n\nSoma da diagonal: {soma}")
```

Exercício 7 — Matriz de notas

```
# Cria uma matriz vazia
notas = []

# Temos 4 alunos
for i in range(4):

    # Cria uma lista para as notas do aluno
    aluno = []

    # Cada aluno possui 3 avaliações
    for j in range(3):

        # Pede a nota
        nota = float(
            input(
                f"Nota do aluno {i + 1}, "
                f"avaliação {j + 1}: "
            )
        )

        # Adiciona a nota à lista do aluno
        aluno.append(nota)

    # Adiciona as notas do aluno à matriz
    notas.append(aluno)

print("\nMédias dos alunos")

# Percorre os 4 alunos
for i in range(4):

    # Soma as três notas
    soma = sum(notas[i])

    # Calcula a média
    media = soma / 3

    # Mostra a média do aluno
    print(f"Aluno {i + 1}: Média = {media:.2f}")
```

Exercício 8 — Struct Produto

```
# Importa o dataclass
from dataclasses import dataclass

# Cria a estrutura Produto
@dataclass
class Produto:

    # Nome do produto
    nome: str

    # Código do produto
    codigo: int

    # Preço do produto
    preco: float

    # Quantidade disponível
    quantidade: int

# Cria uma lista para armazenar os produtos
produtos = []

# Cadastra 5 produtos
for i in range(5):

    print(f"\nProduto {i + 1}")

    # Recebe os dados
    nome = input("Nome: ")
    codigo = int(input("Código: "))
    preco = float(input("Preço: "))
    quantidade = int(input("Quantidade: "))

    # Cria um objeto Produto
    produto = Produto(
        nome,
        codigo,
        preco,
        quantidade
    )

    # Adiciona o produto à lista
    produtos.append(produto)
```

```
produtos.append(produto)

# Começa considerando o primeiro produto
maior_produto = produtos[0]

print("\nProdutos cadastrados")

# Percorre todos os produtos
for produto in produtos:

    # Calcula o valor total daquele produto no estoque
    valor_estoque = produto.preco * produto.quantidade

    # Mostra os dados
    print("\nProduto")
    print(f"Nome: {produto.nome}")
    print(f"Código: {produto.codigo}")
    print(f"Preço: R$ {produto.preco:.2f}")
    print(f"Quantidade: {produto.quantidade}")
    print(f"Valor em estoque: R$ {valor_estoque:.2f}")

    # Calcula o valor do produto que atualmente é o maior
    maior_valor = (
        maior_produto.preco *
        maior_produto.quantidade
    )

    # Verifica se o produto atual possui maior valor
    if valor_estoque > maior_valor:

        # Atualiza o maior produto
        maior_produto = produto

# Calcula o valor do produto com maior estoque
valor_maior = (
    maior_produto.preco *
    maior_produto.quantidade
)
```

```
# Mostra o resultado
print("\nProduto com maior valor em estoque")
print(f"Produto: {maior_produto.nome}")
print(f"Valor: R$ {valor_maior:.2f}")
```

Exercício 9 — Struct Aluno

```
# Importa o dataclass
from dataclasses import dataclass

# Cria a estrutura Aluno
@dataclass
class Aluno:

    # Nome do aluno
    nome: str

    # Idade do aluno
    idade: int

    # Primeira nota
    nota1: float

    # Segunda nota
    nota2: float

    # Terceira nota
    nota3: float

    # Cria um método para calcular a média
    def media(self):

        # Soma as três notas e divide por 3
        return (
            self.nota1 +
            self.nota2 +
            self.nota3
        ) / 3

# Cria uma lista para armazenar os alunos
alunos = []
```

```
# Cadastra 5 alunos
for i in range(5):

    print(f"\nAluno {i + 1}")

    # Recebe os dados do aluno
    nome = input("Nome: ")
    idade = int(input("Idade: "))
    nota1 = float(input("Nota 1: "))
    nota2 = float(input("Nota 2: "))
    nota3 = float(input("Nota 3: "))

    # Cria o objeto Aluno
    aluno = Aluno(
        nome,
        idade,
        nota1,
        nota2,
        nota3
    )

    # Adiciona o aluno à lista
    alunos.append(aluno)

# Contador de aprovados
aprovados = 0

# Contador de reprovados
reprovados = 0

# Considera o primeiro aluno como o de maior média
maior_aluno = alunos[0]

print("\nResultado dos alunos")

# Percorre todos os alunos
for aluno in alunos:
```

```
# Calcula a média
media = aluno.media()

# Verifica se o aluno foi aprovado
if media >= 7:

    # Define a situação
    situacao = "Aprovado"

    # Aumenta o contador
    aprovados += 1

else:

    # Define a situação
    situacao = "Reprovado"

    # Aumenta o contador
    reprovados += 1

# Mostra os dados do aluno
print(
    f"{aluno.nome} - "
    f"Média: {media:.2f} - "
    f"{situacao}"
)

# Verifica se ele possui uma média maior
if media > maior_aluno.media():

    # Atualiza o aluno com maior média
    maior_aluno = aluno

# Mostra o resumo
print("\nResumo")
print(f"Aprovados: {aprovados}")
print(f"Reprovados: {reprovados}")

# Mostra o aluno com maior média
print("\nAluno com maior média")
```

```
print(f"Nome: {maior_aluno.nome}")  
print(f"Média: {maior_aluno.media():.2f}")
```

Exercício 10 — Sistema Integrado

```
# Importa o dataclass  
from dataclasses import dataclass  
  
# Cria a estrutura Funcionario  
@dataclass  
class Funcionario:  
  
    # Nome do funcionário  
    nome: str  
  
    # Idade do funcionário  
    idade: int  
  
    # Cargo do funcionário  
    cargo: str  
  
    # Salário do funcionário  
    salario: float  
  
# Cria uma lista para armazenar os funcionários  
funcionarios = []  
  
# Função responsável por cadastrar os funcionários  
def cadastrar_funcionarios():  
  
    # Repete o cadastro 10 vezes  
    for i in range(10):  
  
        print(f"\nFuncionário {i + 1}")  
  
        # Solicita os dados  
        nome = input("Nome: ")  
        idade = int(input("Idade: "))  
        cargo = input("Cargo: ")  
        salario = float(input("Salário: "))
```



```
# Cria um objeto Funcionario
funcionario = Funcionario(
    nome,
    idade,
    cargo,
    salario
)

# Adiciona o funcionário à lista
funcionarios.append(funcionario)

# Informa que o cadastro terminou
print("\nFuncionários cadastrados!")

# Função para listar os funcionários
def listar_funcionarios():

    # Verifica se a lista está vazia
    if len(funcionarios) == 0:

        print("\nNenhum funcionário cadastrado.")

    # Encerra a função
    return

print("\nFuncionários")

# Percorre todos os funcionários
for funcionario in funcionarios:

    # Mostra os dados
    print("\nFuncionário")
    print(f"Nome: {funcionario.nome}")
    print(f"Idade: {funcionario.idade}")
    print(f"Cargo: {funcionario.cargo}")
    print(f"Salário: R$ {funcionario.salario:.2f}")

# Função para encontrar o maior salário
def maior_salario():
```

```
# Verifica se existem funcionários
if len(funcionarios) == 0:

    print("\nNenhum funcionário cadastrado.")
    return

# Começa considerando o primeiro funcionário
maior = funcionarios[0]

# Percorre todos os funcionários
for funcionario in funcionarios:

    # Compara os salários
    if funcionario.salario > maior.salario:

        # Atualiza o funcionário com maior salário
        maior = funcionario

# Mostra o resultado
print("\nMaior salário")
print(f"Nome: {maior.nome}")
print(f"Cargo: {maior.cargo}")
print(f"Salário: R$ {maior.salario:.2f}")

# Função para calcular a média salarial
def media_salarial():

    # Verifica se existem funcionários
    if len(funcionarios) == 0:

        print("\nNenhum funcionário cadastrado.")
        return 0

    # Começa a soma em zero
    soma = 0

    # Percorre todos os funcionários
    for funcionario in funcionarios:
```

```
# Adiciona o salário à soma
soma += funcionario.salario

# Divide a soma pela quantidade de funcionários
media = soma / len(funcionarios)

# Mostra a média
print("\nMédia salarial")
print(f"Média: R$ {media:.2f}")

# Retorna a média
return media

# Função para mostrar funcionários acima da média
def acima_da_media():

    # Verifica se existem funcionários
    if len(funcionarios) == 0:

        print("\nNenhum funcionário cadastrado.")
        return

    # Calcula a soma dos salários
    soma = 0

    # Percorre os funcionários
    for funcionario in funcionarios:

        soma += funcionario.salario

    # Calcula a média salarial
    media = soma / len(funcionarios)

    print("\nFuncionários acima da média")

    # Percorre os funcionários novamente
    for funcionario in funcionarios:
```

```
# Verifica se o salário está acima da média
if funcionario.salario > media:

    # Mostra o funcionário
    print(
        f"{funcionario.nome} - "
        f"R$ {funcionario.salario:.2f}"
    )

# Loop principal do sistema
while True:

    # Mostra o menu
    print("\nSistema de Funcionários")
    print("1 - Cadastrar funcionários")
    print("2 - Listar funcionários")
    print("3 - Maior salário")
    print("4 - Média salarial")
    print("5 - Funcionários acima da média")
    print("0 - Sair")

    # Pede uma opção
    opcao = input("\nEscolha uma opção: ")

    # Verifica a opção escolhida
    if opcao == "1":

        cadastrar_funcionarios()

    elif opcao == "2":

        listar_funcionarios()

    elif opcao == "3":

        maior_salario()

    elif opcao == "4":
```

```
media_salarial()

elif opcao == "5":

    acima_da_media()

elif opcao == "0":

    # Encerra o programa
    print("\nPrograma encerrado.")
    break

else:

    # Caso a opção não exista
    print("\nOpção inválida.")
```